

PROJECT DOCUMENTATION

Documentation - 2

NO-CODE BUSINESS PREDICTOR

using AWS SageMaker Canvas

Predicting Mobile App User Churn with Cloud-Native No-Code ML

Technology Stack

AWS SageMaker Canvas
Amazon S3
AWS Lambda
Amazon SNS

Document Details

Type: Project Report
Domain: Cloud ML / No-Code AI
Category: Applied Machine Learning

1. Project Flow Overview

This project builds a cloud-based machine learning pipeline to predict which mobile app users are likely to stop using the platform — a problem known as churn. Rather than writing custom ML code, the solution uses AWS SageMaker Canvas, a no-code AutoML tool, combined with Amazon S3, AWS Lambda, and Amazon SNS to automate the full cycle from data to alert.

The pipeline runs in five stages. First, historical user behaviour data is cleaned and uploaded to Amazon S3, which acts as the central storage layer for everything — raw data, model outputs, and inference results. Second, SageMaker Canvas pulls the data from S3 and handles model training automatically, testing multiple algorithms and picking the best one without any manual coding.

Third, once training is done, the model runs a batch prediction job on the current user base and writes the results — churn labels and confidence scores — back to S3. Fourth, an AWS Lambda function picks up this output the moment it lands in S3 and filters out the highest-risk users. Fifth, it fires alerts to an Amazon SNS topic, so the right teams get notified and can act quickly.

The design is intentionally modular. Each component talks to the others only through S3 events or SNS messages, which makes it easy to scale, swap out parts, or extend the pipeline later. The goal throughout is to turn raw behavioural data into retention actions without any manual handoffs in between.

Objective: Identify users likely to churn within the next billing cycle and automatically alert retention teams so they can take action before it's too late.

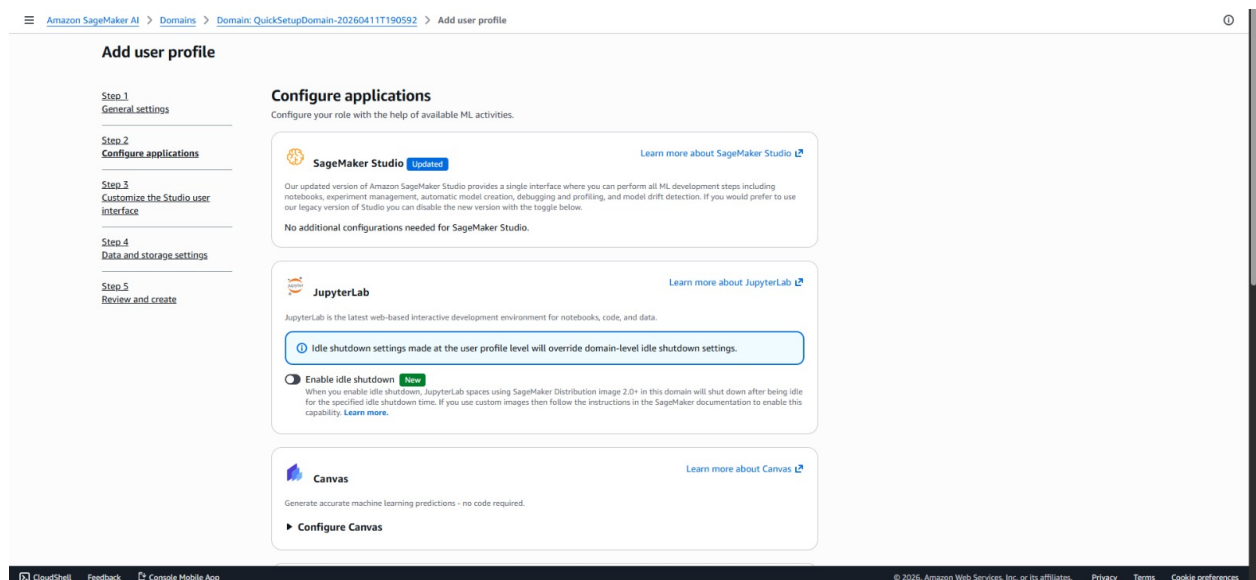
2. Step-by-Step Implementation

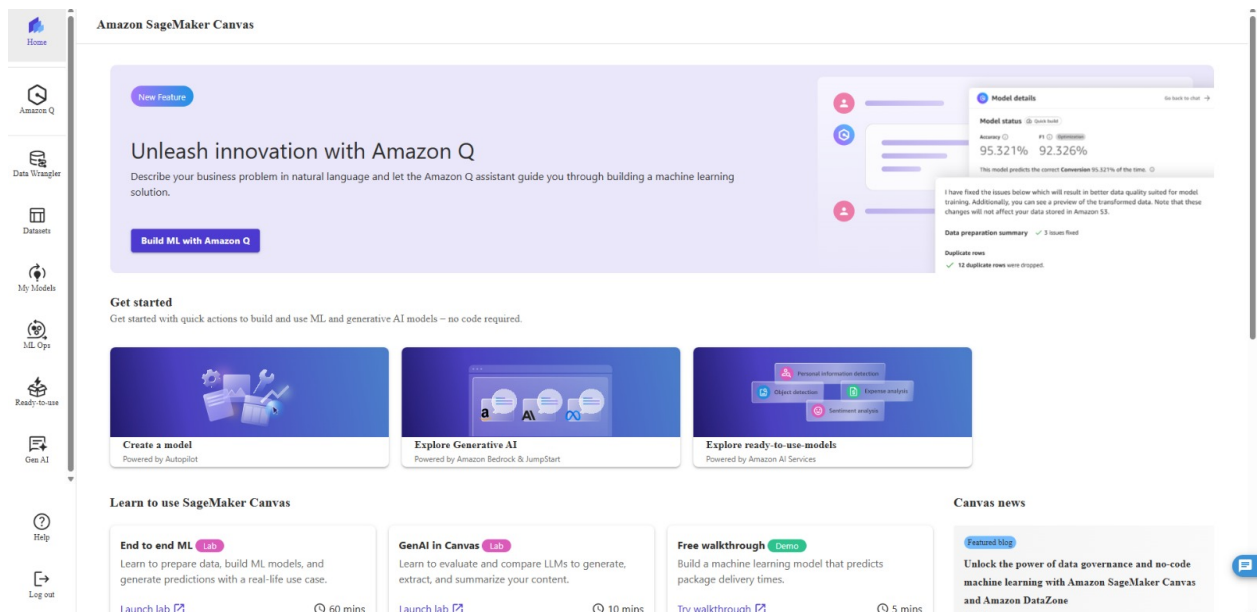
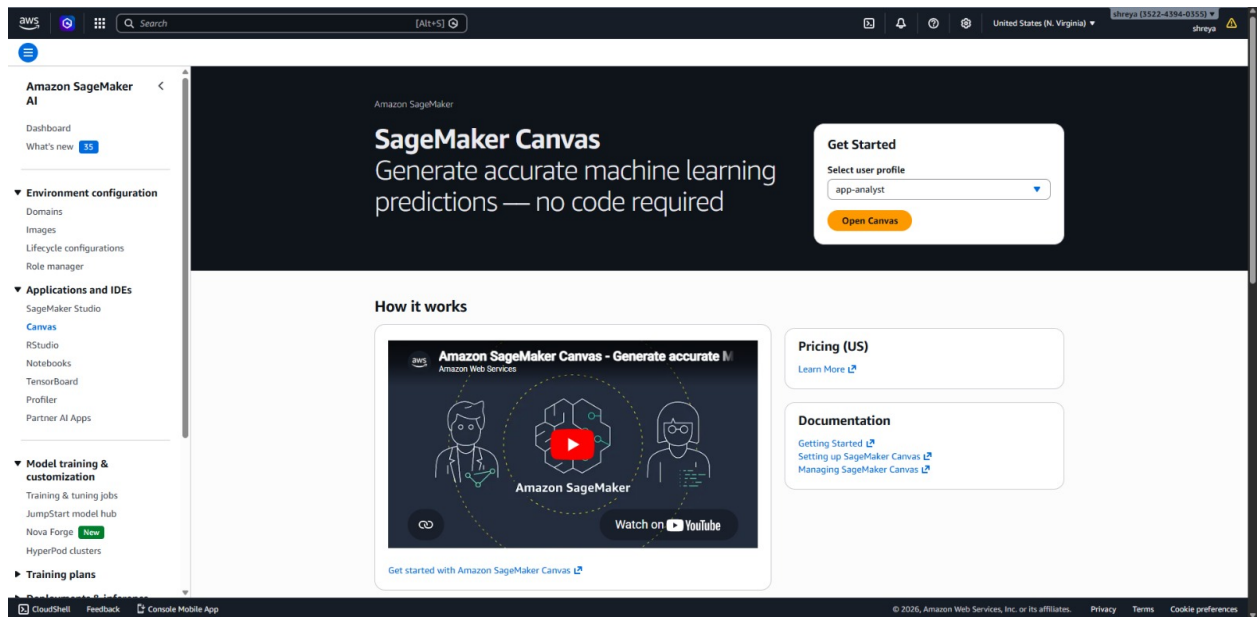
2.1 Data Preparation

Start by collecting user engagement data from the app's backend — things like session frequency, average session length, in-app purchases, support tickets, last login date, subscription tier, and device platform. Clean the data by filling in missing values, removing duplicates, and making sure the churn label column (0 or 1) is correct. Export it as a CSV and upload it to an S3 bucket, for example `churn-prediction-data-bucket/training-data/`. Enable S3 versioning so you always have a history of dataset changes.

2.2 Configuring AWS SageMaker Canvas

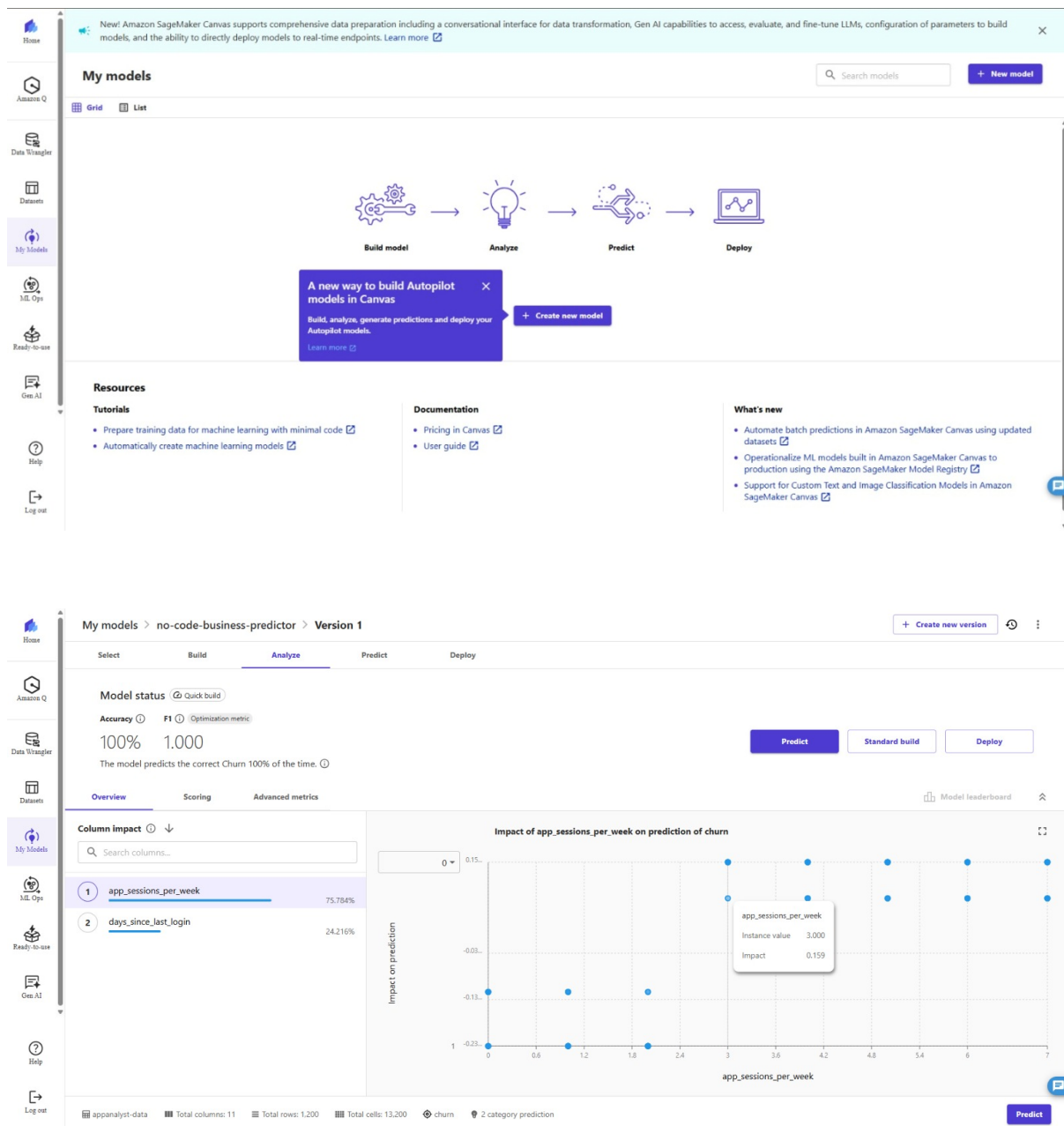
Open SageMaker Canvas from the AWS Console and set up a domain with an IAM execution role that has read/write access to your S3 bucket. Inside Canvas, create a new dataset by pointing it to your S3 training file. Canvas will automatically detect column types and show data statistics. Set the churn column as the target variable, choose binary classification, and kick off the model build. Use Standard Build for production — it takes longer but tests far more algorithms and gives much better results.





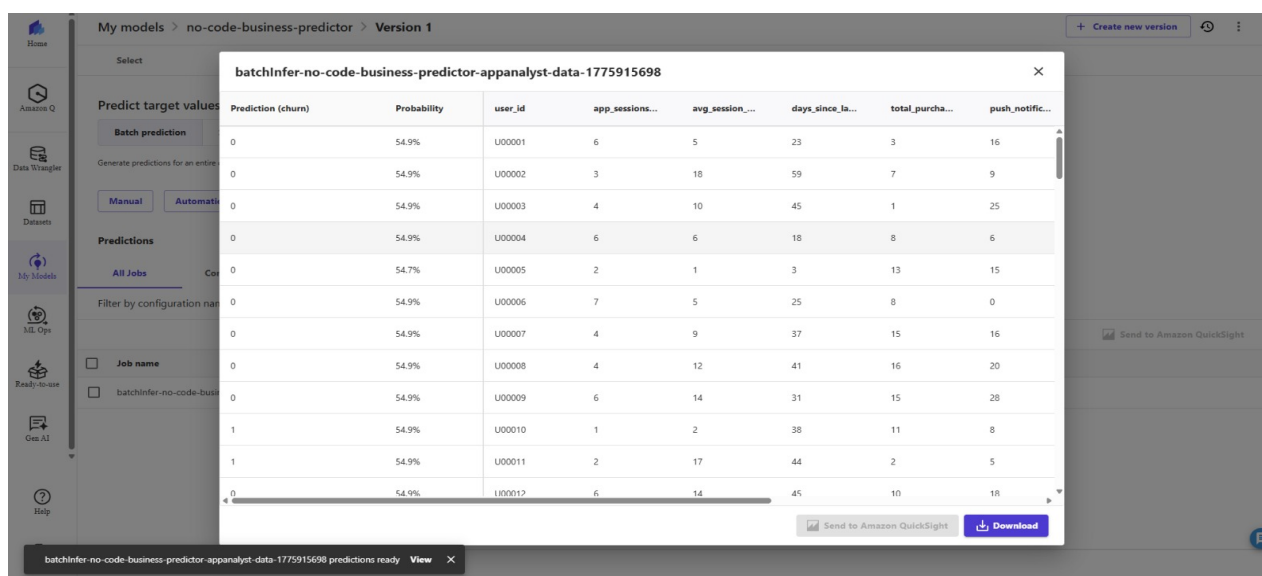
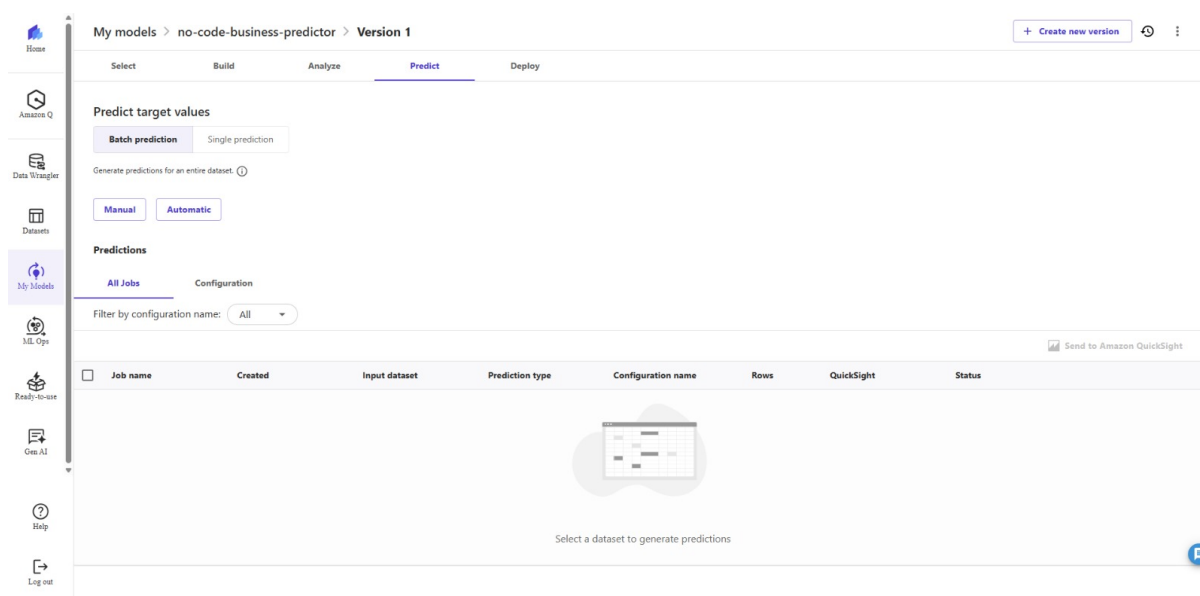
2.3 Model Training and Evaluation

SageMaker Canvas runs AutoML behind the scenes using SageMaker Autopilot. It tries several algorithm families, applies feature engineering, and tunes hyperparameters automatically. When it's done, it shows a ranked leaderboard of models sorted by F1 score. Review the confusion matrix, AUC-ROC curve, precision, and recall to confirm the model is performing well. Pay attention to the feature importance chart too — it shows which user behaviours (like declining session counts) are the biggest churn signals. Approve the best model before moving on.



2.4 Generating Batch Predictions

Upload the current active user dataset (without the churn column) to S3 under inference-input/. In Canvas, select the approved model and run a batch prediction against this file. Canvas processes every record and outputs a new CSV to inference-output/ containing two extra columns: the predicted churn label (0 or 1) and the probability score (0.0–1.0). This file is what triggers the alerting pipeline next.



2.4 Generating Single Prediction

Once the model is trained, SageMaker Canvas lets you test it against a single user in real time — no file uploads needed. Go to My Models, open the churn model, click the Predict tab, select Single Prediction, fill in that user's details, and Canvas instantly returns a churn label (0 or 1) along with a confidence score.

The screenshot shows the AWS SageMaker Canvas interface for a model named 'no-code-business-predictor' at 'Version 1'. The 'Predict' tab is active. On the left, a table lists input features: user_id (U00008), app_sessions_per_week (2), avg_session_duration_min (4), days_since_last_login (24), total_purchases (3), push_notification_clicks (5), support_tickets_raised (2), and subscription_type (Free). On the right, the 'churn Prediction' section displays a large 'Yes' result. Below it, a legend indicates 'New prediction' (blue bar) and 'Last prediction' (grey bar). A horizontal bar chart shows the probability for 'No' (45.058%) and 'Yes' (54.942%). A 'Download prediction' button is at the bottom right.

Column	Value
user_id	U00008
app_sessions_per_week	2
avg_session_duration_min	4
days_since_last_login	24
total_purchases	3
push_notification_clicks	5
support_tickets_raised	2
subscription_type	Free

churn Prediction

Yes

■ New prediction
■ Last prediction

No 45.058%
Yes 54.942%

Download prediction

The screenshot shows the AWS SageMaker Canvas interface for the same model. The input features are: user_id (U00001), app_sessions_per_week (0), avg_session_duration_min (15), days_since_last_login (20), total_purchases (16), push_notification_clicks (16), support_tickets_raised (2), and subscription_type (Free). The 'churn Prediction' section displays a large 'No' result. The horizontal bar chart shows the probability for 'No' (54.757%) and 'Yes' (45.243%).

Column	Value
user_id	U00001
app_sessions_per_week	0
avg_session_duration_min	15
days_since_last_login	20
total_purchases	16
push_notification_clicks	16
support_tickets_raised	2
subscription_type	Free

churn Prediction

No

■ New prediction
■ Last prediction

No 54.757%
Yes 45.243%

Download prediction

2.5 AWS Lambda Automation and SNS Alerting

A Lambda function (Python 3.11) is set up with an S3 trigger on the inference-output/ prefix. The moment Canvas drops the prediction file there, Lambda fires automatically. It downloads the file using boto3, reads it row by row, and flags anyone with a churn probability above the threshold (default: 0.75). It then publishes those flagged users to an SNS topic. Subscribers — email, SMS, or webhooks — get the alert within seconds. If anything goes wrong during processing, errors are logged to CloudWatch and a dead-letter queue catches failed invocations.

The screenshot shows the AWS Lambda console for the 'churnalert-function'. A green banner at the top states: 'Successfully created the function "churnalert-function". You can now change its code and configuration. To invoke your function with a test event, choose "Test".'

Function overview

- Diagram: Template
- Function name: churnalert-function
- Layers: (0)
- Buttons: + Add trigger, + Add destination
- Buttons: Throttle, Copy ARN, Actions
- Buttons: Export to Infrastructure Composer, Download

Code source

- Buttons: Open in Visual Studio Code, Upload from
- Code editor showing `lambda_function.py`:

```
def lambda_handler(event, context):
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = event['Records'][0]['s3']['object']['key']
    print("Processing file: ", key)
    response = s3.get_object(Bucket=bucket, Key=key)
    lines = response['Body'].read().decode('utf-8-sig').splitlines()
    reader = csv.DictReader(lines)
```

Function details:

- Function ARN: `arn:aws:lambda:us-east-1:352243940355:function:churnalert-function`
- Description: -
- Last modified: 1 minute ago

Tutorials sidebar:

- Info
- Tutorials
- Create a simple web app
- In this tutorial you will learn how to:

 - Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
 - Invoke your function through its function URL

- Learn more
- Start tutorial

The screenshot shows the 'Add trigger' configuration page for the 'churnalert-function'. A green banner at the top states: 'Policy was successfully attached to role.'

Trigger configuration

- Trigger type: S3 (aws, asynchronous, storage)
- Bucket: `s3/sagemaker-us-east-1-352243940355`
- Bucket region: us-east-1
- Event types: Select the events that you want to have trigger the Lambda function. You can optionally set up a prefix or suffix for an event. However, for each bucket, individual events cannot have multiple configurations with overlapping prefixes or suffixes that could match the same object key.
- Prefix - optional: Enter a single optional prefix to limit the notifications to objects with keys that start with matching characters. Any special characters must be URL encoded. (e.g. `images/`)
- Suffix - optional: Enter a single optional suffix to limit the notifications to objects with keys that end with matching characters. Any special characters must be URL encoded. (e.g. `.jpg`)
- Recursive invocation: If your function writes objects to an S3 bucket, ensure that you are using different S3 buckets for input and output. Writing to the same bucket increases the risk of creating a recursive invocation, which can result in increased Lambda usage and increased costs. [Learn more](#)
- ☐ I acknowledge that using the same S3 bucket for both input and output is not recommended and that this configuration can cause recursive invocations, increased Lambda usage, and increased costs.
- Lambda will add the necessary permissions for AWS S3 to invoke your Lambda function from this trigger.

Tutorials sidebar:

- Info
- Tutorials
- Create a simple web app
- In this tutorial you will learn how to:

 - Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
 - Invoke your function through its function URL

- Learn more
- Start tutorial

The screenshot shows the AWS IAM console for the 'churnalert-function-role-vxukho42' role. A green banner at the top states: 'Policy was successfully attached to role.'

Role details:

- Creation date: April 11, 2026, 19:35 (UTC+05:30)
- ARN: `arn:aws:iam:352243940355:role/service-role/churnalert-function-role-vxukho42`
- Last activity: -
- Maximum session duration: 1 hour

Permissions

- Permissions policies (3)
- You can attach up to 10 managed policies.
- Filter by Type: All types
- Table of permissions policies:

Policy name	Type	Attached entities
AmazonS3FullAccess	AWS managed	2
AmazonSNSFullAccess	AWS managed	2
AWSLambdaBasicExecutionRole-b14294d2-5780-46ea-8424-4...	Customer managed	1

Permissions boundary (not set)

Generate policy based on CloudTrail events

- You can generate a new policy based on the access activity for this role, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)
- Buttons: Generate policy
- No requests to generate a policy in the past 7 days.

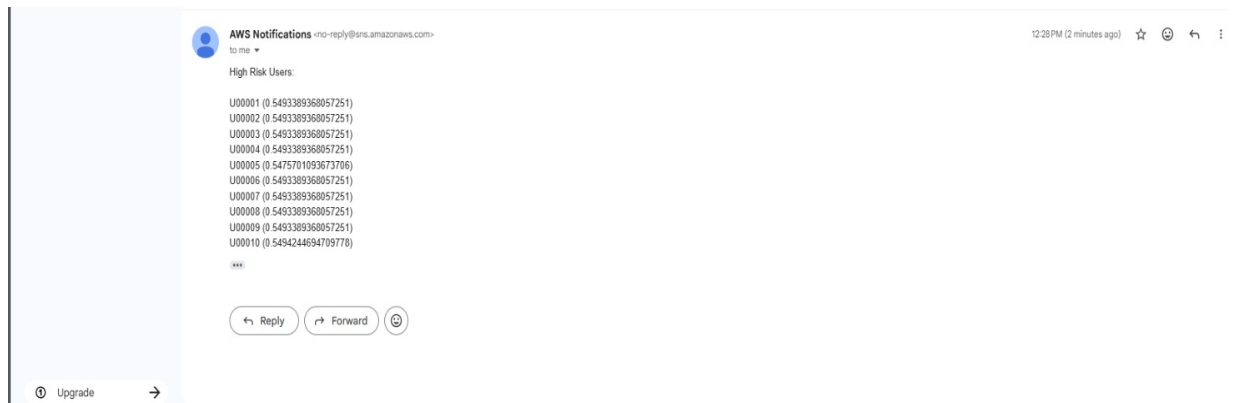
Tutorials sidebar:

- Info
- Tutorials
- Create a simple web app
- In this tutorial you will learn how to:

 - Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
 - Invoke your function through its function URL

- Learn more
- Start tutorial

OUTPUT:



3. Flow Between Steps

System Data Flow — Step Summary

Step 1	Data Preparation — Cleaned user CSV uploaded to S3 /training-data/ — the starting point for everything.
Step 2	Canvas Dataset Import — SageMaker Canvas reads from S3, profiles columns, and prepares the dataset for training.
Step 3	AutoML Training — Canvas runs Standard Build AutoML — tests algorithms, tunes hyperparameters, picks the winner.
Step 4	Model Evaluation — Review confusion matrix, AUC-ROC, F1 score, and feature importance before approving the model.
Step 5	Batch Prediction — Model scores the active user dataset; output CSV with churn labels and probabilities saved to S3.
Step 6	Lambda Trigger — S3 ObjectCreated event fires automatically, invoking the Lambda function within seconds.
Step 7	SNS Alert Dispatch — Lambda filters high-risk users and publishes alerts to SNS — teams are notified immediately.

4. Error Handling and Use Cases

Any system running in production needs to handle things going wrong gracefully. This pipeline has error handling built in at every layer — from bad input data to Lambda crashes to notification failures. It also turns out the core architecture is flexible enough to support a few other useful business scenarios beyond churn prediction.

4.1 Data Validation Errors

If the training CSV has mismatched column types — say a numeric column accidentally contains text because of an upstream pipeline bug — SageMaker Canvas will throw a schema error on import. To catch this early, a validation script runs before the upload. It checks for type mismatches, flags any column where more than 10% of values are missing, and confirms the churn label is strictly 0 or 1. If any check fails, the upload is blocked and the error details are written to a dedicated S3 error log folder.

4.2 Model Training Failures

Canvas needs at least 500 labelled records to train reliably. Below that, the model quality drops and Canvas may warn you. A bigger problem is class imbalance — if churned users are less than 5% of the dataset, the model can look accurate on paper but completely miss real churners. The fix is to apply SMOTE (Synthetic Minority Over-sampling) before uploading, which artificially balances the training set so the model actually learns what churn looks like.

4.3 Lambda Execution Errors

Lambda has a 5-minute timeout. If the prediction file is very large, the function processes records in batches of 1,000 rows at a time to stay within limits. If the prediction file is corrupt or incomplete — maybe the inference job crashed partway through — the CSV parser will throw an exception. This is caught with a try-except block that logs the error to CloudWatch and sends a secondary alert to an ops SNS topic so the engineering team knows immediately. Any Lambda invocations that fail after all retries get routed to a Dead-Letter Queue (SQS) for manual inspection.

4.4 SNS Delivery Failures

SNS guarantees at-least-once delivery, meaning it will retry if a subscriber endpoint is unreachable. Email and HTTP endpoints use exponential backoff retry policies. If an endpoint keeps failing beyond the threshold, SNS logs the event to CloudWatch. With a subscription-level DLQ configured, any undelivered messages are stored in SQS for later recovery rather than being silently dropped.

4.5 Error Reference Table

Error Type	Root Cause	Resolution
Schema Mismatch	Incompatible types or malformed CSV columns	Pre-upload validation script with strict schema checks
Insufficient Data	Training set under 500 records	Apply SMOTE oversampling; extend historical data window
Lambda Timeout	Prediction file too large for one invocation	Paginated batch processing + Dead-Letter Queue (DLQ)
SNS Delivery Fail	Subscriber endpoint unreachable	Exponential backoff retry + subscription-level DLQ
Class Imbalance	Churn class < 5% of total dataset	Oversample minority class; track F1 and AUC-ROC closely

4.6 Extended Use Cases

The same pipeline can be adapted for subscription renewal forecasting — just relabel the churn column to represent non-renewals and rerun the flow. The probability scores also work well as segmentation inputs for A/B testing: split users into high, medium, and low risk tiers and test different retention messages across each group. Finally, combining churn probability with monthly revenue gives you a Customer Lifetime Value (CLV) estimate, which helps prioritise which users are worth spending the most effort to retain.

5. Key Features Implemented

The system was designed with production use in mind, not just as a proof of concept. Each feature below addresses a real operational concern — cost, security, reliability, or usability — and together they make the pipeline genuinely deployable in a business setting.

5.1 No-Code AutoML via SageMaker Canvas

SageMaker Canvas handles the entire ML development cycle — data profiling, feature engineering, algorithm selection, hyperparameter tuning, and evaluation — through a graphical interface. No training code needed. The underlying AutoML engine (Autopilot) has been tested against standard benchmarks and performs competitively with hand-crafted models on structured tabular data. This makes the system accessible to teams without dedicated data scientists.

5.2 Serverless Automation via AWS Lambda

Lambda runs the alerting logic with zero always-on infrastructure. You only pay per invocation, which makes it very cost-efficient for a pipeline that runs periodically rather than continuously. The function is configured with 512 MB of memory and a 5-minute timeout, tuned based on actual test runs with realistic file sizes. The S3 event trigger means the whole process is hands-free — no cron jobs or manual triggers needed.

5.3 Multi-Channel Alerts via Amazon SNS

The SNS topic has multiple subscriber types configured: email for product managers, SMS for on-call customer success staff, and HTTPS webhooks for CRM integrations like Salesforce or HubSpot. A single Lambda publish call fans out to all of them simultaneously. This means the right people get notified through the right channel without setting up separate notification systems for each one.

5.4 Configurable Risk Threshold

The churn probability threshold — defaulting to 0.75 — is stored as a Lambda environment variable, not hardcoded. Operations teams can adjust it without touching the code or redeploying. It can also be overridden per-invocation via the event payload, which is useful for end-of-quarter campaigns where you may want to cast a wider net and capture more at-risk users even at the cost of some false positives.

5.5 Monitoring via CloudWatch

Every Lambda invocation writes structured logs to CloudWatch: records processed, high-risk users flagged, execution duration, and any error details. CloudWatch alarms notify the engineering team if error rates spike or execution time creeps toward the timeout limit. S3 access logging is also enabled on the prediction bucket to maintain a full audit trail for compliance purposes.

5.6 IAM Least-Privilege Security

Every AWS service in the pipeline runs with a tightly scoped IAM role. SageMaker Canvas can only read from and write to the specific S3 prefixes it needs. The Lambda function can only read the `/inference-output/` prefix and publish to the designated SNS topic — nothing else. All policies are documented and version-controlled alongside the project, making security audits straightforward.

Conclusion

This project shows that you don't need to write complex ML code to build something genuinely useful in production. By combining SageMaker Canvas, S3, Lambda, and SNS, the system covers the full cycle — from raw user data to actionable churn alerts — with minimal custom development and no infrastructure to manage.

The architecture is built to last. Because each component communicates through S3 events and SNS messages rather than direct calls, individual parts can be updated or replaced without touching the rest of the system. The built-in error handling, CloudWatch monitoring, and dead-letter queues mean it's designed to fail gracefully and recover quickly when things go wrong.

More broadly, this project reflects where ML is heading: managed, accessible, and integrated directly into cloud infrastructure. As organisations look to apply predictive analytics to retention, risk, and operations, architectures like this one — event-driven, serverless, and no-code where possible — are becoming the practical standard for teams that need results without a large data science function.